

RV-MalTrace: Hardware-Assisted Behavior Tracing for RISC-V Linux Workloads

Anonymous Authors

Abstract—Malware behavior analysis often relies on software tracing, such as syscall logs or emulator traces. These tools are useful, but they also become part of the software environment being measured. Hardware-assisted tracing provides a separate observation path, but raw hardware events do not directly answer security questions. They must be connected to executable identity, process context, and reference logs without confusing one evidence source for another.

We present RV-MalTrace, a RISC-V Linux tracing system that connects a CVA6/Genesys2 hardware trace path with an offline behavior reconstruction pipeline. The system records control-flow, trap, syscall, context, marker, and pointer-byte events. It then combines those events with ELF metadata and runtime process maps, while marking whether each reconstructed field came from hardware trace, the executable, the runtime map, or a reference log such as QEMU or strace.

The current implementation supports controlled RISC-V Linux workloads and a reproducible artifact package rooted at `results/evaluation/genesys2-cva6/current/`. The main result is therefore scoped: RV-MalTrace can support source-labeled behavior reconstruction for controlled workloads when the required trace, executable, runtime-map, and reference inputs are present. The evaluation covers trace ordering, syscall pairing, source labeling, code attribution, and local reproduction. It does not claim real-malware accuracy, payload equivalence, production streaming/DMA throughput, JTAG RAM boot, SD-card-free kernel update, or cycle-level board overhead.

I. INTRODUCTION

Malware behavior analysis often starts with observable events: which files a program opens, whether it creates child processes, when it calls `execve`, and whether it reaches unusual traps or memory-permission changes. Software tracers can collect these events, but they run inside the environment being studied. A workload may detect the tracer, change behavior because of tracing overhead, or force the analyst to trade detail for lower perturbation.

Hardware-assisted tracing gives the analyst another observation point. It can record architectural events below the operating system and the traced process. However, raw events are not enough. A program counter needs an executable and load address. A syscall transition needs a process context. A path or expected syscall sequence may come from a reference run rather than from hardware. A paper that mixes these sources risks overstating what the hardware recovered.

RV-MalTrace addresses this problem for controlled RISC-V Linux workloads. The system collects hardware trace records from a CVA6-based platform, joins them with ELF metadata and runtime process maps, and marks the source of each reconstructed field. The result is a behavior trace whose evidence is explicit: the reader can see which parts came from hardware and which parts came from supporting metadata or reference logs.

The main finding in this draft is deliberately narrow. RV-MalTrace supports controlled behavior reconstruction when the question is whether a trace record sequence can be connected to syscall transitions, executable identity, process context, and clearly labeled reference evidence. The supporting evidence is organized around trace correctness, source labeling, code attribution, and reproducibility. Each result is reported with the boundary needed to interpret it: local code-attribution fixtures do not become board-native source-line evidence, reference logs do not become hardware output, and controlled workloads do not establish in-the-wild malware accuracy.

This paper is scoped as a systems and artifact paper. It is not a malware classifier and does not measure family-level accuracy, indicator coverage, or uncontrolled malware execution. It also does not evaluate production trace transport, cycle-level overhead, or JTAG RAM boot. The contribution is a reproducible RISC-V hardware tracing pipeline for controlled behavior studies with clear evidence sources.

We make the following contributions:

- A CVA6/Genesys2 trace path for controlled RISC-V Linux behavior studies, with trace records for retire, trap, syscall entry/return, privilege transition, branch/jump, context, marker, and pointer-byte snapshots.
- A behavior reconstruction pipeline that keeps hardware trace fields separate from ELF metadata, runtime process-map evidence, and reference logs, so that derived fields carry explicit source labels.
- Code-attribution checks for ELF identity, PIE/ASLR load bias, dynamic objects, fork/exec ownership, stripped binaries, and source-map boundaries, reported as local attribution evidence rather than board-native source-line recovery.
- A reproducible artifact package rooted at `results/evaluation/genesys2-cva6/current/`, with manifests and SHA-256 path checks that keep supported results separate from experiments that are not evaluated in this draft.

II. BACKGROUND

A. RISC-V Linux Trace Context

RISC-V exposes architectural privilege transitions, traps, branches, jumps, and system-call boundaries that can be useful for behavioral analysis. On a CVA6-class core, these events can be observed near instruction retirement and then packaged into records consumed by an offline analyzer [1], [2]. The resulting events are lower-level than strace-style syscall logs: they can include trap and control-flow context, but they also require careful reconstruction before they become security-relevant behavior.

B. Why Source Labels Matter

A behavioral summary can combine multiple evidence sources. A syscall entry record may be hardware-derived. The function or binary that owns a PC may come from an exact ELF and its load ranges. The current process identity may come from a runtime process map. QEMU, strace, or host-control logs may provide a reference for expected behavior. Treating these fields as one undifferentiated trace would overstate what the hardware alone recovered.

RV-MalTrace therefore records source labels as part of the reconstruction output. A summary is used in this draft only when the source of each field is explicit and the supporting artifact path is recorded under the current evidence directory.

III. THREAT MODEL AND SCOPE

This draft studies controlled RISC-V Linux workloads on a CVA6/Genesys2 trace path. The target workload runs under an experiment harness that marks the start and end of the measured region. The attacker model is limited to software behavior visible through the collected architectural events and supporting runtime metadata.

The paper does not evaluate resistance against a malicious workload that can physically modify the board, reprogram the trace logic, or compromise the host collection machine. It also does not evaluate malware-family detection, indicators of compromise, or TTP coverage. Safe synthetic and malware-like workloads are used to test trace and reconstruction behavior; they are not reported as uncontrolled real-malware validation.

IV. DESIGN

RV-MalTrace has three stages: hardware collection, offline reconstruction, and reproducibility checking. The collection stage records architectural events. The reconstruction stage joins those records with executable and runtime context. The checking stage verifies that reported results are backed by files in the artifact package.

A. Trace Records

The CVA6 adapter emits event records for retire, trap, syscall entry, syscall return, privilege transition, branch, jump, context, pointer-byte snapshots, and markers. The evaluated configuration requires strict SRET qualification so that user-mode return evidence is not widened by a permissive timing rule.

Markers define the measured workload window. The trace buffer clears on a begin marker, stops on an end marker, and records the begin marker as the first event in the window. This lets the analysis distinguish the intended workload from boot or background activity.

B. Source Labels

The design avoids a common failure mode in trace analysis: reporting a reconstructed field without identifying its source. RV-MalTrace labels fields as hardware trace, ELF metadata, runtime map, or reference log. A field that depends on QEMU, strace, or host-control data can validate expected behavior, but it is not treated as hardware output.

V. IMPLEMENTATION

The implementation has three components.

CVA6 trace adapter. The adapter converts execution and control signals into a compact event stream. The directed trace-correctness fixtures exercise trap/retire separation, strict syscall entry/return pairing, same-cycle ordering, dual-commit ordering, privilege transitions, and negative cases for unqualified SRET events.

Genesys2 trace path. The current board evidence uses a BRAM trace buffer for controlled marker windows. Production streaming or DMA transport is not evaluated in this draft; it requires separate measurements for transport design, clocking, host reception, parsing, drop accounting, timing, resource use, and interference.

Offline reconstruction tools. The Python tooling parses trace records, builds ELF code maps, joins trace PCs to code ranges, recovers process and file/path summaries where evidence permits, and emits machine-readable summaries. The artifact checks require each reported result to point to a file in the evidence directory and to match its recorded SHA-256 digest.

VI. BEHAVIOR RECONSTRUCTION

RV-MalTrace reconstructs behavior in layers. Hardware records provide the event sequence and architectural context. ELF metadata provides binary identity and code ranges. Runtime process maps bind the target process and its loaded objects. Reference logs provide expected behavior for comparison but are not reported as hardware output.

The local code-attribution fixtures cover cases that commonly break simple PC-to-binary mapping: ELF SHA-256 identity, PIE and ASLR load bias, dynamic library ownership, fork/exec ownership, stripped binaries, and source maps not captured on the board. Source-line information is reported only when it is bound to the exact board artifact.

This reconstruction model is intentionally limited. It supports evidence such as syscall pairing, measured-window target attribution, and executable ownership where the required inputs exist. It does not evaluate a complete OS process graph, complete filesystem history, complete memory-object tracking, or recovery of all pointer strings.

TABLE I
SCOPE OF THE CVA6/GENESYS2 DRAFT.

Area	Evidence Used	Scope
Artifact root	Manifests and hashes	The draft uses <code>results/evaluation/genesys2-cva6/current/</code> as the current evidence directory.
Trace correctness	Directed tests	Syscall entry/return pairing, trap/retire separation, and event ordering are evaluated.
Source labels	Reconstruction summaries	Fields identify whether they came from hardware trace, ELF metadata, runtime maps, or reference logs.
Code attribution	Local tests	ELF identity, PIE/ASLR load bias, dynamic objects, fork/exec, and stripped binaries are covered.
Production streaming/DMA	Not evaluated	Throughput and interference measurements require a separate transport study.
JTAG RAM boot	Not evaluated	The current observed target does not expose a Vivado memory-control object.
Real malware	Not evaluated	Safe malware-like workloads are not reported as uncontrolled real-malware validation.

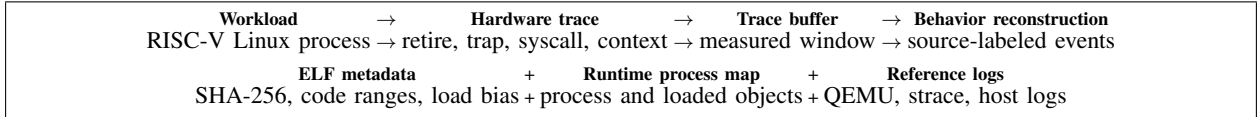


Fig. 1. RV-MalTrace pipeline. Hardware trace records are joined with executable metadata and runtime context. Reference logs are used for comparison, not reported as hardware output.

Source labels	
Hardware	event kind, PC, trap cause, marker order
ELF	binary hash, section range, symbol range
Runtime map	process identity, load object ownership
Reference log	expected syscall or path behavior

Fig. 2. Each reconstructed field records where it came from.

VII. EVALUATION

The evaluation asks whether the implementation can support controlled behavior studies from hardware trace records. We evaluate trace correctness, source labeling, code attribution, and reproducibility. Board transport throughput, cycle overhead, and RAM boot are outside the evaluated scope.

The evaluation is organized as claim, evidence, and boundary rather than as a log of experiments. A passing check supports only the claim named in Table II. It does not automatically support neighboring claims, such as detector accuracy, payload equivalence, production transport throughput, or board-native source-line attribution.

A. RQ1: Trace Correctness

The directed tests check event ordering and syscall pairing at the trace-record level. They cover trap/retire separation, strict syscall entry/return pairing, same-cycle ordering, dual-commit ordering, privilege transitions, and negative relaxed-SRET cases.

B. RQ2: Source Labeling

The source-labeling checks verify that reconstructed fields identify their source: hardware trace, ELF metadata, runtime map, or reference log. Reference traces can validate expected behavior, but they are not reported as hardware-recovered fields.

C. RQ3: Code Attribution

The local code-analysis tests cover ELF identity, PIE/ASLR load bias, dynamic object attribution, fork/exec ownership, stripped binaries, and source-map boundaries. These tests support attribution logic, but they are not reported as a new board run or as board-native DWARF evidence.

D. RQ4: Reproducibility

The artifact package is organized around a canonical evidence directory and machine-readable manifests. Quick local reproduction is:

```
uv run rvmt repro:quick
```

Docker-local reproduction is:

```
uv run rvmt ndss:docker-full
```

The current project status records successful quick, local, full, and Docker-local reproduction paths. Experiments that were not run are recorded separately and are not included in the results.

E. Evidence Boundaries and Alternative Explanations

The current evidence rules out several implementation failures, but it does not remove all alternative explanations. First, a reconstruction may appear semantically rich because reference logs supply expected behavior. RV-MalTrace therefore labels reference-derived fields and does not report them as hardware recovery. Second, local code-attribution fixtures check the analysis logic, but they do not prove board-native source-line recovery. Third, controlled workloads exercise selected behavior patterns; they do not establish malware-family accuracy, indicator coverage, or payload equivalence. Fourth, resource and timing reports describe implementation cost and timing closure; they do not measure cycle-level runtime slowdown.

TABLE II
EVALUATION QUESTIONS, SUPPORTING EVIDENCE, AND BOUNDARIES.

RQ	Claim tested	Evidence	Result	Boundary
RQ1	Trace records preserve ordering and syscall pairing needed for reconstruction	Directed trace tests	Covered by the current test corpus	Does not measure production transport throughput or runtime slowdown
RQ2	Reconstructed fields identify whether they came from hardware trace, ELF metadata, runtime maps, or reference logs	Reconstruction summaries	All reported fields carry a source label	Reference-derived fields validate expectations but are not hardware recovery
RQ3	Code attribution handles common mapping edge cases	Local attribution tests	Attribution edge cases are covered locally	Local fixtures do not establish board-native source-line recovery
RQ4	The evidence package is reproducible and auditable	Local and Docker runs	Quick, local, full, and Docker reproduction complete	Reproduction covers the recorded artifact, not experiments outside the manifest
RQ5	Controlled workloads exercise behavior reconstruction without becoming a malware benchmark	Case-study manifests and behavior checks	Controlled behavior checks are recorded in the artifact package	Does not imply real-malware accuracy, TTP coverage, or payload equivalence

VIII. LIMITATIONS

The current implementation is a research artifact, not a production tracing system.

First, the current evaluation does not include production streaming/DMA throughput. That result requires a separate transport study.

Second, board cycle-overhead is not evaluated. Existing probes show that the required live cycle-source path is not yet available in the current board state.

Third, JTAG RAM boot and SD-card-free kernel update are not evaluated. The observed Vivado Hardware Manager target exposes the device and ILA path but no memory-control or hart-control object for RAM loading.

Fourth, real malware validation is not evaluated. Safe synthetic and malware-like workloads test trace and reconstruction behavior under controlled conditions; they do not establish malware-family accuracy, IOC coverage, TTP coverage, or payload equivalence.

ETHICS CONSIDERATIONS

The current paper scope avoids uncontrolled malware execution. The repository uses controlled safe synthetic and malware-like workloads to exercise trace and behavior reconstruction. Any future real-malware experiment must define containment, sample lineage, release policy, and artifact-sanitization procedures before it is cited as paper evidence.

The artifact separates public summaries from raw board and host logs. Hashes, manifests, and sanitized summaries support reproducibility without requiring unrestricted publication of every raw artifact.

IX. RELATED WORK

Software tracing and reference runs. Tools such as strace observe syscall behavior from within the software environment, while QEMU can provide emulation-based execution logs [3], [4]. RV-MalTrace uses such sources only as references. Its contribution is the separation between reference-derived fields and hardware trace evidence.

Kernel instrumentation. eBPF-based tracing provides powerful kernel observability and is widely used for production diagnostics and security monitoring [5]. It remains

software-visible and kernel-mediated. RV-MalTrace is complementary: it studies the evidence boundary of a hardware-assisted RISC-V trace path rather than replacing kernel instrumentation.

RISC-V hardware trace. RISC-V systems and open-source cores such as CVA6 make it practical to prototype trace logic at the architectural boundary. The challenge addressed here is not only trace capture, but behavior reconstruction with explicit evidence sources for security workloads.

X. CONCLUSION

RV-MalTrace demonstrates a path from RISC-V hardware trace records to controlled Linux behavior summaries. The current CVA6/Genesys2 implementation supports trace correctness tests, source-labeled reconstruction, local code-attribution tests, and reproducible artifact checks under the current evidence directory. The supported conclusion is that hardware-assisted trace can be connected to behavior reconstruction under controlled inputs while preserving the source of each derived field. Real malware validation, production transport, JTAG RAM boot, SD-card-free kernel update, and cycle-overhead are outside the evaluated scope.

The next revision should make two decisions explicit. First, the submission claim should remain behavior reconstruction with source-labeled evidence rather than malware detection accuracy. Second, the paper should choose which remaining gap is most important to close before submission: board-native source-line evidence, cycle-overhead measurement, or production trace transport. Remaining prose-level summaries should then be replaced with final figures, quantitative tables, and related-work positioning under the same scope.

REFERENCES

- [1] RISC-V International, “The RISC-V Instruction Set Manual,” <https://riscv.org/technical/specifications/>, 2024.
- [2] OpenHW Group, “CVA6 RISC-V CPU,” <https://github.com/openhwgroup/cva6>, 2026.
- [3] The strace Developers, “strace,” <https://strace.io/>, 2026.
- [4] F. Bellard, “QEMU, a Fast and Portable Dynamic Translator,” in *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, 2005.
- [5] The Linux Kernel Documentation Project, “BPF Documentation,” <https://docs.kernel.org/bpf/>, 2026.